

Summary of changes from BSIM-CMG_112.0.0 to BSIM-CMG_112.1.0

BSIM Group, UC Berkeley

A. Pampori (pampori@berkeley.edu), Shivendra Singh Parihar, and
Abhishek Kumar

A. Summary of Bugs

1. 2024bug10(Synopsys): Improved Vdsat Implementation to prevent floating point limitations
2. 2025bug1 (Intel): Define separate length_scaling macro for CV parameters.
3. 2025bug2 (ADI): Prevent divide-by-zero by QSREF_i
4. 2025bug3 (Samsung): Update documentation to remove ETAQM.
5. 2025bug4 (ADI): Multiple fixes for HVMOD=1
6. 2025bug5 (ADI): Parameter limits depending on instance parameters
7. 2025bug6 (ADI): Add error conditions for LeffCV/LeffCV_acc and Lg
8. 2025bug7 (Synopsys): Correct TFIN dependence for Ies
9. 2025bug8 (ADI): Make binned parameters `MPRnb with limit checks on the final binned variable
10. 2026bug1 (Synopsys): Fix node assignment for RGATEMOD=2
11. 2026bug2 (Intel): Multiple fixes for FNMOD=2

B. Summary of Enhancements

1. 2024enh11(ADI): Using functions instead of macros for lln, lexp, and hypsmooth
2. 2025enh1 (ADI, QA subcommittee): Extended QA to evaluate non-default parameter values.
3. 2025enh2 (Intel): Improved gate resistance model
4. 2025enh3 (GF): Improve capability to accurately model capacitance in FinFETs/GAAFETs with LDD
5. 2025enh4 (Intel, ADI): Improvements in documentation

6. 2025enh5 (Intel): Adjust limits to muhc0 and muhc1
7. 2025enh6 (ADI): Update QA suite to prevent noise output clipping below threshold levels.
8. 2025enh7 (Intel): Improve the flicker noise model to offer flexible dependence on mobility/QM parameters

C. Description of Bug fixes

1. 2024bug10(Synopsys): Improved Vdsat Implementation to prevent floating point limitations

Issue: V_{dsat} calculation involves a term $Tb - \sqrt{Tb * Tb - 2.0 * Ta * Tc}$. However, under certain conditions, $Tb \approx \sqrt{Tb * Tb - 2.0 * Ta * Tc}$.
e.g.

Tb= 1.0803493634730794
 $\sqrt{Tb * Tb - 2.0 * Ta * Tc}$ = 1.0803493634726788
 Difference= 4.0056846728475648e-13

This leads to a situation where V_{dsat} remains unchanged with changing bias.

Solution: Use the identity $a - b = \frac{a^2 - b^2}{a + b}$

While $a^2 - b^2$ still suffers from a loss of precision, the presence of a fully precise $a + b$ in the denominator ensures that the bias dependence of V_{dsat} remains intact, while maintaining backward compatibility.

2. 2025bug1 (Intel): Define separate length_scaling macro for CV parameters.

The length_scaling macro, defined using Leff, was being used for CV parameters like PSATCV and VSATCV.

```
// Length scaling
#define length_scaling(param, a_term, b_term) \
begin \
    param = param + a_term * `lexp(-Leff / b_term); \
end
```

This version introduces a new macro length_scaling_CV to be used for CV parameters.

3. 2025bug2 (ADI): Prevent divide-by-zero by QSREF_i

bsimcmg_checking.include has been updated to include the following check to prevent a divide-by-zero error by QSREF_i:

```
if (QSREF_i <= 0.0) begin
    $error("Fatal: Effective qsref = %g is not positive.", QSREF_i);
End
```

4. 2025bug3 (Samsung): Update documentation to remove ETAQM.

BSIM-CMG 108.0.0

```
// *** Toxeff model for quantum mechanical effects ***
// ** Normal operation (Vgs > Vfb) **
if(QMTCENCU_i > 0.0 || QMTCENCU_i > 0.0) begin
    T4 = (qia + ETAQM * qba) / QM0;
    T5 = 1.0 + pow(T4, PQM);
    Tcen = Tcen0 / T5;
```

BSIM-CMG 109.0.0

```
// Toxeff model for quantum mechanical effects
// Normal operation (Vgs > Vfb)
if (QMTCENCU_i > 0.0) begin
    T4 = qia / QM0;
    T5 = 1.0 + pow(T4, PQM);
    Tcen = Tcen0 / T5;
    coxeff = 1.0 / (1.0 / (cox * EOT / TOXP) + Tcen * QMTCENCU_i / epssub);
end else begin
    coxeff = cox;
end
```

(Doc. in BSIM-CMG 112.0.0beta4)

BSIM-CMG 112.0.0beta4 Technical Manual
3.7 Quantum Mechanical Effects

3.7.1 Charge Centroid Calculation for Inversion

If $QMTCENCU_i > 0$ then

$$T4 = \frac{q_{ia} + ETAQM \cdot q_{ba}}{QM0}$$

$$T5 = 1 + T4^{PQM}$$

$$T_{cen} = \frac{T_{cen0}}{T5}$$

end

The documentation was updated to reflect the latest code.

5. 2025bug4 (ADI): Multiple fixes for HVMOD=1

- Fix node assignment:
 $T2 = \text{devsign} * V(\text{gi}, \text{di1}) - \text{vfbsd_v}; > T2 = \text{devsign} * V(\text{gi}, \text{di2}) - \text{vfbsd_v};$
- Remove devsign from vdr_ov:
 $\text{vdr_ov} = \text{devsign} * \text{abs}(V(\text{di1}, \text{si})); > \text{vdr_ov} = \text{abs}(V(\text{di1}, \text{si}));$
- Use conductance instead of resistance:
 $I(\text{di1}, \text{di}) <+ V(\text{di1}, \text{di}) / 1\text{e-}3; > I(\text{di1}, \text{di}) <+ V(\text{di1}, \text{di}) * 1\text{e}3;$
- Include devsign in capacitance contributions:
 $I(\text{ge}, \text{di1}) <+ \text{ddt}(\text{qbov}); > I(\text{ge}, \text{di1}) <+ \text{devsign} * \text{ddt}(\text{qbov});$
 $I(\text{ge}, \text{di}) <+ \text{ddt}(\text{qbov_s}); > I(\text{ge}, \text{di}) <+ \text{devsign} * \text{ddt}(\text{qbov_s});$
- Remove HVMOD conditional from operating point variables qg, qd, and qs.

6. 2025bug5 (ADI): Parameter limits depending on instance parameters

The lower limit for the model parameter deltaprds depends on the instance parameter fpitch. The limit has been removed, and an error message has been included for the case when deltaprds is lower than -fpitch.

```
if (deltaprds_v < -fpitch) begin
    $error("Fatal: The value of deltaprds = %g is less than
- %g (-fpitch).", deltaprds, fpitch);
    deltaprds_v = 0.0;
end
```

7. 2025bug6 (ADI): Add error conditions for LeffCV/LeffCV_acc and Lg

For all variables like Lg, Leffl, WGAAeffl, etc, that can cause numerical issues through a divide-by-zero, invalid log, etc, add an error condition for values ≤ 0 and reassign the variable to a safe value after the \$error flag.

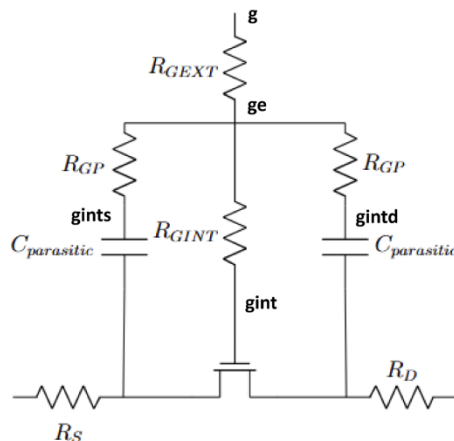
8. 2025bug7 (Synopsys): Correct TFIN dependence for Ies

Bias-dependent sidewall junction currents to use TFIN instead of Weff0, following 2015bug16 for the bias-independent counterparts.

9. 2025bug8 (ADI): Make binned parameters 'MPRnb with limit checks on the final binned variable

Binning parameters (e.g. lqsref) for all binned parameters, like qsref, noia, etc., now use the 'MPRnb macro. Further, all binned parameters now use the 'MPRnb macro with the appropriate limits being enforced using \$error conditions.

10. 2026bug1 (Synopsys): Fix node assignment for RGATEMOD=2



gint was found to be floating in 112.1.0beta0_3. This has been corrected with the following change:

```
// NQS gate resistance model
if (nqsmode == 1 && XRCRG1_i != 0.0) begin
    I(gint, gi) <+ V(gint, gi) * gcrg;
end else begin
    V(gint, gi) <+ 0.0;
end
```

11. 2026bug2 (Intel): Multiple fixes for FNMOD=2

Following Intel's suggestions, NOIA was changed to NOIAeff and Coxeff to Cox in the FNMOD=2 implementation.

D. Description of Enhancements

1. 2024enh11(ADI): Using functions instead of macros for `lln`, `lexp`, and `hypsmooth`

The following definitions in `bsimcmg_macros.include` were replaced by their respective functions in `bsimcmg_body.include`:

```
// // Clamped exponential
// `define lexp(x) ((x > `EXPL_THRESHOLD) ? `MAX_EXPL * (1.0 + x -
`EXPL_THRESHOLD) : (x < -`EXPL_THRESHOLD) ? `MIN_EXPL : exp(x))

// // Clamped logarithm
// `define llnf(x) ((x > `N_MINLOG) ? ln(x) : `LN_N_MINLOG)

// // Hyperbolic smoothing
// `define hypsmooth(x, c) ((x < -1e4 * (c)) ? (-(c) * (c) / (x)) : (0.5 *
(x + sqrt((x) * (x) + 4.0 * (c) * (c)))))

// // Hyperbolic smoothing with a maximum value
// `define hypmax(x, xmin, c) (xmin + 0.5 * (x - (xmin) - c + sqrt((x -
(xmin) - c) * (x - (xmin) - c) - 4.0 * (xmin) * c)))

// // Smoothing with a minimum value
// `define smoothminx(x, x0, deltax) (0.5 * (x + x0 + sqrt((x - (x0)) * (x
- (x0)) + 0.25 * deltax * deltax)))

// // Smoothing with a maximum value
// `define smoothmaxx(x, x0, deltax) (0.5 * (x + x0 - sqrt((x - (x0)) * (x
- (x0)) + 0.25 * deltax * deltax)) + 0.25 * deltax)

// // Smoothing with a maximum value
// `define smoothmaxx2(x, x0, deltax) (0.5 * (x + x0 - sqrt((x - (x0)) *
(x - (x0)) + 0.25 * deltax * deltax)))
```

```
// //Gamma function
// `define gammafunc(x) (0.0385 * exp(-4.6 * (x-1)) + 7.5893e-7 *
pow(2*(x) - 3, 8) + 6.9583e-5 * pow(2*(x-1), 6) - 6.583e-4 * pow(2*(x-1),
5) + 0.0065 * pow(2*(x-1), 4) - 0.026 * pow(2*(x-1), 3) + 0.1371 *
pow(2*(x-1), 2) - 0.194 * 2*(x-1) + 0.959)
```

2. 2025enh1(ADI, QA subcommittee): Extended QA to evaluate non-default parameter values.

The non-default parameter value file nmosNonDefaultParameters_C/
pmosNonDefaultParameters_C was updated to include the latest set of parameters,
ensuring all parameters are being tested for a non-default condition.

3. 2025enh2 (Intel): Improved gate resistance model

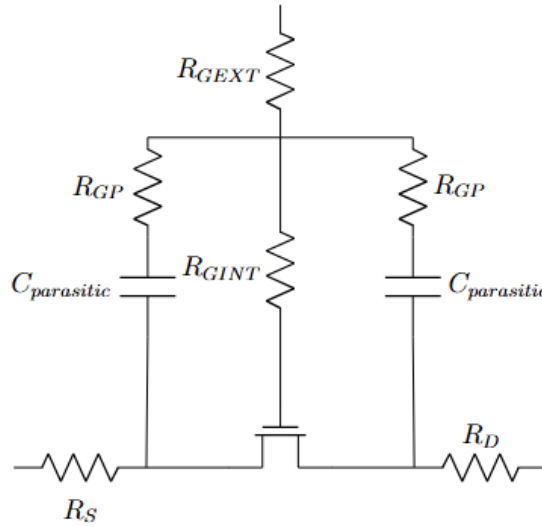


Figure 6: RC Network for RGATEMOD=2

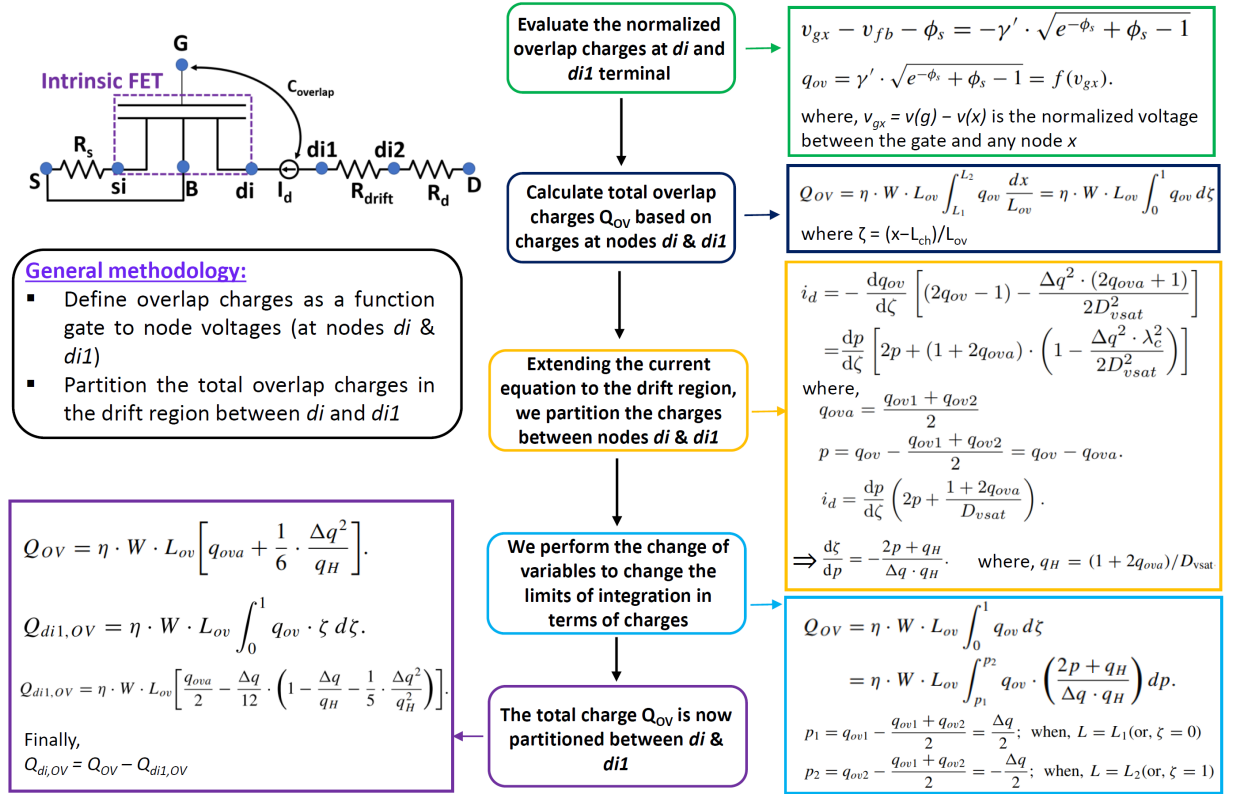
3.15.4.1 Improved gate resistance model for parasitic capacitance charging

This improved model (Fig. 6) can be activated by setting RGATEMOD=2. The model accounts for the impact of the gate work function on the effective voltage applied to the parasitic capacitances. The two resistors (set using the parameter RGP) capture the effective voltage seen by the parasitic capacitances, while RGINTE helps modulate the voltage seen by the gate capacitance.

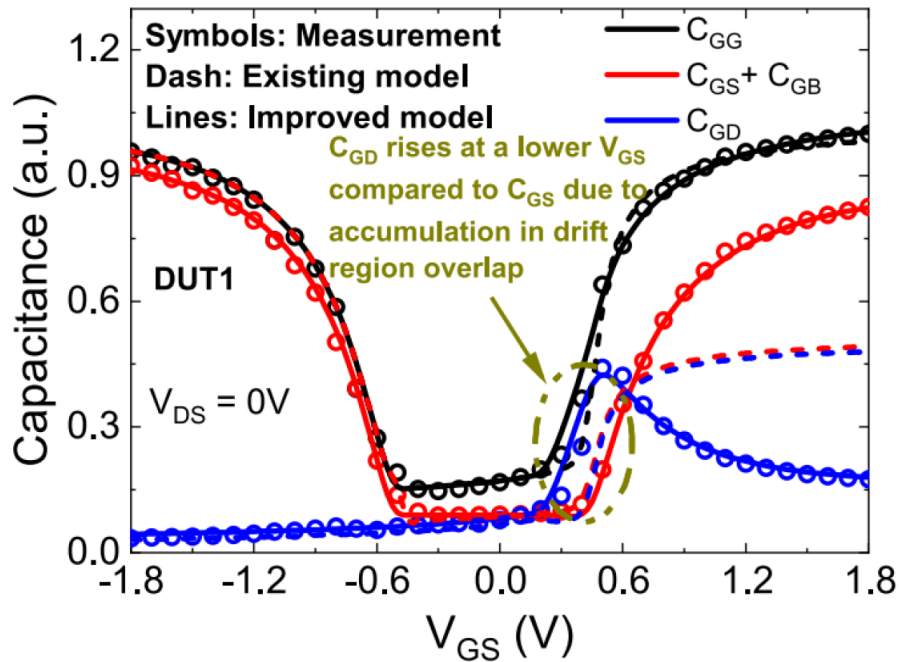
4. 2025enh3 (GF): Improve capability to accurately model capacitance in

FinFETs/GAAFETs with LDD

An improved methodology to model asymmetric capacitance in Fin-LDMOS devices is introduced:



This approach accurately captures the humps observed in C_{gd} :



5. 2025enh4 (Intel, ADI): Improvements in documentation

- Improved the documentation for thermal noise.
- Mathematical functions written in a non-italicized format.
- Some binning equations (3.74 – 3.76) used L (in the manual) instead of Lg (in the code). The manual has now been updated to align with the code.

6. 2025enh5 (Intel): Adjust limits to muhc0 and muhc1

A broader range for parameters muhc0 and muhc1 has been introduced to offer more flexibility in model extraction:

```
`MPRoo(muhc0, 0.0, "", -inf, 1.0, "Coefficient for hot-carrier induced mobility degradation")
```

```
`MPRco(muhc1, 0.0, "", 0.0, inf, "Exponential coefficient for hot-carrier induced mobility degradation")
```

7. 2025enh6 (ADI): Update QA suite to prevent noise output clipping below threshold levels

- PMOS gate noise levels are lower than NMOS due to a difference in the gate current magnitude. Gate noise tests for PMOS have now been scaled (using toxref=2e-9 ntox=15) to maintain an output noise level higher than the threshold levels.

Change test 921 to evaluate drain noise instead of gate noise (both for NMOS and PMOS).

8. 2025enh7 (Intel): Improve the flicker noise model to offer flexible dependence on mobility/QM parameters

- Introduced a new mod (FNMOD=2) that considers a pure number fluctuation model for flicker noise that is highly insensitive to mobility fluctuations¹.

The noise PSD is defined as:

$$S_{id^2} = \left(\frac{q^2 kT}{fW\gamma} \right) \frac{g_m^2 N_T^{\text{eff}}}{C_{\text{ox}}^2 WL} \left(\frac{1}{1 + K0NOI (q_s - q_d)} \right)^{K1NOI} \left(\frac{\log \left(\frac{q_s + \frac{1}{2}}{q_d + \frac{1}{2}} \right) (q_s + q_d + 1)}{2 (q_s - q_d)} \right)$$

¹ A. S. Roy, S. Kim and S. P. Mudanai, "An Improved Flicker Noise Model for Circuit Simulations," in IEEE Transactions on Electron Devices, vol. 64, no. 4, pp. 1689-1694, April 2017, doi: 10.1109/TED.2017.2669153.

where $K0NOI$ and $K1NOI$ are model parameters, $N_T^{\text{eff}} = \text{NOIA} + \text{NOIB} \cdot q_m + \text{NOIC} \cdot q_m^2$ and $q_m = 0.5(q_s + q_d)$.